CS112 — BÀI TẬP LỚN · PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

TÔ MÀU ĐỒ THỊ (BÀI TOÁN SỞ THÚ)

Thiết kế, so sánh năm chiến lược giải và demo minh họa từng bước

---

- Môn: CS112 — Phân tích và Thiết kế Thuật toán, lớp CS112.F31
- Giảng viên: [SV XÁC NHẬN]
- Sinh viên: [SV XÁC NHẬN — Họ tên] · MSSV [SV XÁC NHẬN]
- Demo trực tuyến: algorithm-visualizer.hieutk.dev

Lý do chọn đề tài và nguồn gốc

- Nhận xét của cô sau bài thi: phần thiết kế thuật toán của nhiều bạn còn yếu, dù dạng bài "Sở thú là bài tô màu đồ thị" không mới.
- Gợi ý từ AI: ba mô hình (Claude, Codex/GPT, Grok) cùng đề xuất chọn tô màu đồ thị, vì có nhiều cách giải để so sánh và trạng thái từng bước dễ minh họa.
- [SV XÁC NHẬN] — đây phải là lý do thật của sinh viên; nội dung trên là bản nháp từ lịch sử làm việc.

Nguồn gốc

Nguồn	Ghi chú
Bài toán 4 màu	Guthrie 1852, Appel–Haken chứng minh 1976
Levitin §1.3, §12.1	Giới thiệu & backtracking m màu
Bài Sở thú	Đề thi, buổi ôn tập của môn
Welsh–Powell (1967)	Heuristic theo degree
DSATUR — Brélaz (1979)	Heuristic theo độ bão hòa

Ứng dụng thực tế

Ứng dụng	Ánh xạ sang tô màu đồ thị
Xếp lịch thi	Môn = vertex; hai môn chung sinh viên = edge; ca thi = màu
Sở thú / xếp phòng	Loài/khách kỵ nhau không ở chung chuồng/phòng
Cấp phát thanh ghi (compiler)	Biến "sống" cùng lúc không dùng chung thanh ghi
Phân tần số trạm phát sóng	Hai trạm gần nhau phải khác tần số
Giải Sudoku	Bài toán tô 9 màu trên đồ thị 81 vertex

Phát biểu bài toán (kiểu đề thi / Wecode)

- **Bài toán k-coloring (mặc định):**
- Sở thú cần xếp n loài thú vào các chuồng. Có danh sách các cặp loài "kỵ nhau", không được nhốt chung. Sở thú hiện có k chuồng. Có xếp được tất cả các loài vào k chuồng, sao cho không chuồng nào chứa hai loài kỵ nhau, hay không? Nếu có, hãy chỉ ra cách xếp.
- **Mở rộng — tìm chromatic number $\chi(G)$:**
- Không cho trước k. Hỏi cần ít nhất bao nhiêu chuồng, và chỉ ra một cách xếp dùng đúng số chuồng đó.

Mô hình hóa bằng đồ thị

- Đồ thị vô hướng G = (V, E): mỗi loài là một vertex; mỗi cặp kỵ nhau là một edge; mỗi chuồng là một màu.
- Tô màu hợp lệ: hàm c: V → {0,…,k−1} sao cho (u,v) ∈ E ⇒ c(u) ≠ c(v).
- k-coloring (k ≥ 3) thuộc lớp NP-đầy đủ (Karp, 1972). k = 2 giải được trong thời gian đa thức — tương đương kiểm tra đồ thị hai phía.

Ba khái niệm cần phân biệt

Khái niệm	Câu hỏi
Tô màu hợp lệ	Một cách gán cho trước có đúng không? (checker)
k-coloring	Có tô được bằng k màu không? (mặc định)
Chromatic number χ(G)	Số màu nhỏ nhất là bao nhiêu? (chromatic)

Input, Output và ràng buộc

- **Input (JSON, định nghĩa bằng Zod):**
- { nodes, edges, colors, mode? } — nodes quyết định thứ tự duyệt của greedy/backtracking/exhaustive search.
- **Output:**
- { found, coloring, heuristic?, colorsUsed?, chromaticExact? } — heuristic báo "chưa tìm được" không khẳng định vô nghiệm.

Ràng buộc chính

Đại lượng	Giá trị
Số vertex n	0 ≤ n ≤ 320
Số edge m	≤ 8.000
Số màu k	1 ≤ k ≤ 10 (chỉ k-coloring)
Backtracking	n ≤ 9 (k-coloring), ≤ 10 (χ)
Exhaustive search	k^n ≤ 200.000 (k-coloring); Σk^n ≤ 200.000 (χ, ~n ≤ 6)

Ví dụ Input / Output

#	Đồ thị	k	Output đúng
1	Tam giác A–B–C	3	found=true, ví dụ A=0, B=1, C=2
2	Tam giác A–B–C	2	found=false (chính xác: vô nghiệm; heuristic: chưa tìm được)
3	Đồ thị vương miện 6 vertex (ví dụ chung)	2	found=true, coloring=[0,1,0,1,0,1] (greedy/WP báo chưa tìm được)
4	Đồ thị rỗng, n = 0	1	found=true, coloring=[]
5	colors = 0	—	Bị từ chối vì dữ liệu không hợp lệ

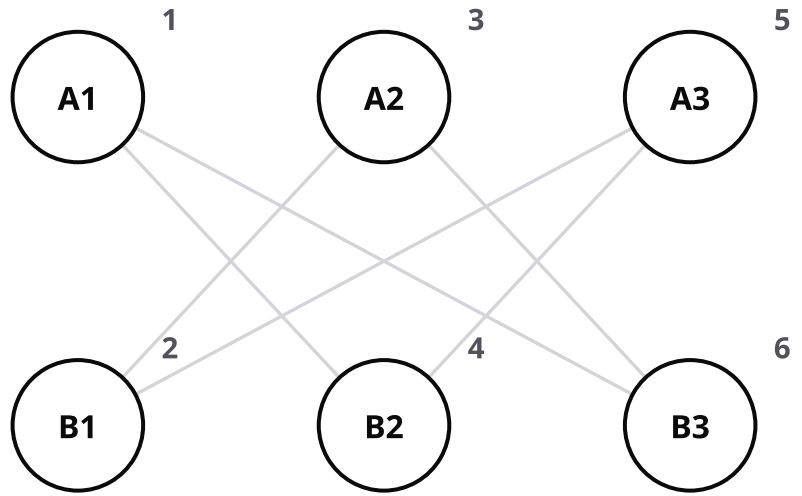
Cấu trúc dữ liệu

- Danh sách vertex có thứ tự (nodes) — quyết định thứ tự duyệt.
- Tập edge dạng Set<string> các cặp vertex — kiểm tra kề nhau trong O(1).
- Mảng màu color[v], với −1 là chưa tô.
- Tập màu bị cấm của vertex v: các màu đang có ở vertex kề v.
- Độ bão hòa sat(v) (chỉ DSATUR): số màu khác nhau ở các vertex kề v.
- Trace từng bước: lineId mã giả, trạng thái đồ thị, state-space tree (treeNodes), lời giải thích.

Phân tích đặc trưng — ý tưởng giải

- Lower bound: một clique kích thước ω cần ít nhất ω màu (tam giác cần 3 màu).
- Upper bound: mọi đồ thị tô được bằng Δ+1 màu (Δ = degree lớn nhất).
- Thứ tự duyệt quyết định chất lượng greedy — luôn có thứ tự cho đúng χ(G), nhưng cũng có thứ tự xấu.
- Ưu tiên vertex "khó" trước (degree cao / bị ràng buộc nhiều) để tránh hết màu.
- Không gian tìm kiếm có k^n cách gán; backtracking pruning sớm để duyệt ít hơn exhaustive search.

Thứ tự xấu cho greedy



Đồ thị vương miện 6 vertex, χ=2 — thứ tự xen kẽ A1,B1,A2,B2,A3,B3: ở k-coloring (k=2) greedy bị bế tắc; ở chromatic mode greedy dùng 3 màu thay vì 2.

Thuật toán 1–2: Greedy và Welsh–Powell

- **Greedy:**
- Xét vertex theo thứ tự cho sẵn; gán màu nhỏ nhất chưa có ở vertex kề; không có màu → "chưa tìm được".
- **Welsh–Powell:**
- order ← vertex sắp theo degree giảm dần, rồi chạy Greedy(order). Trên vương miện mọi vertex cùng degree 2 → thứ tự không đổi → gặp đúng khó khăn của greedy.
- **Độ phức tạp (lý thuyết / trong app):**
- Greedy: $O(n \cdot k + m)$ / $O(n \cdot (n + k))$. Welsh–Powell: $O(n \log n + n \cdot k + m)$ / $O(n^2 + n \cdot k)$.

Greedy — pseudocode.ts (nguyên văn)

```
1. Xét các đỉnh theo thứ tự cho trước
2. Gán màu nhỏ nhất trong k màu chưa
   dùng ở các đỉnh kề đã tô
3. Nếu không còn màu nào khả dụng: dừng
   — heuristic chưa tìm được, không
   chứng minh vô nghiệm
4. Trả về cách tô nếu tô được hết mọi
   đỉnh
```

Welsh–Powell — pseudocode.ts (nguyên văn)

```
1. Sắp các đỉnh theo bậc giảm dần
2. Gán màu nhỏ nhất trong k màu chưa
   dùng ở các đỉnh kề đã tô
3. Nếu không còn màu nào khả dụng: dừng
   — heuristic chưa tìm được, không
   chứng minh vô nghiệm
4. Trả về cách tô nếu tô được hết mọi
   đỉnh
```

Thuật toán 3: DSATUR (Brélaz, 1979)

- Mở rộng ngoài sách — chọn vertex chưa tô có độ bão hòa sat(v) lớn nhất; hòa thì chọn degree lớn hơn.
- Tính chất đã biết: DSATUR luôn tô đúng bằng 2 màu trên đồ thị hai phía → thành công trên ví dụ chung, trong khi greedy thất bại.
- Độ phức tạp lý thuyết $O(n^2+m)$; trong app $O(n^3+n \cdot k)$ vì mỗi vòng tính lại độ bão hòa bằng cách quét mọi cặp vertex.

Mã giả

```
1. sat(v) ← 0 cho mọi vertex
2. Chọn vertex chưa tô có sat(v) lớn
   nhất; hòa → degree lớn hơn
3. Gán màu nhỏ nhất chưa có ở vertex kề
4. Không còn màu: dừng — chưa tìm
   được, không chứng minh vô nghiệm
5. Trả về cách tô nếu tô được hết
```

Thuật toán 4: Quay lui (Backtracking, Levitin §12.1)

- Đúng đắn: chỉ mở rộng cách tô bộ phận hợp lệ → trả về false sau khi duyệt hết nghĩa là chắc chắn vô nghiệm.
- Dừng ở nghiệm đầu tiên (bài toán quyết định k màu).
- State-space tree có ≤ $(n+1)\cdot k^n$ nút. Cận lý thuyết $O((n+1)\cdot k^n\cdot(\Delta+1))$; trong app $O(n^2\cdot k^n)$ vì quét mọi vertex mỗi nút.
- Chế độ $\chi(G)$: lower bound = clique greedy, upper bound = DSATUR, tăng k dần tới nghiệm đầu tiên.

Mã giả

```
Backtrack(i):
  if i = n: return true
  v ← order[i]
  for c in 0..k-1:
    if không vertex kề nào màu c:
      color[v] ← c
      if Backtrack(i+1): return true
      color[v] ← chưa tô  // backtrack
  return false
```

Thuật toán 5: Vét cạn (Exhaustive search)

- Sinh mọi k^n phép gán theo thứ tự từ điển; dừng ở nghiệm đầu tiên hợp lệ.
- Cận lý thuyết $O(k^n \cdot (n+m))$; trong app $O(k^n \cdot n^2)$ vì kiểm tra bằng quét mọi cặp vertex.
- Giới hạn an toàn trong app: $k^n \le 200.000$ (k-coloring) hoặc $\Sigma k^n \le 200.000$, k=1..n (χ, tức $n \le 6$).
- Mốc để so sánh "chi phí pruning" của backtracking (20 bước so với 69 bước trên ví dụ chung).

Mã giả

```
Exhaustive(G, k):
  for mỗi c in {0..k-1}^n
      (thứ tự từ điển):
    if c hợp lệ: return c
  return "vô nghiệm"
```

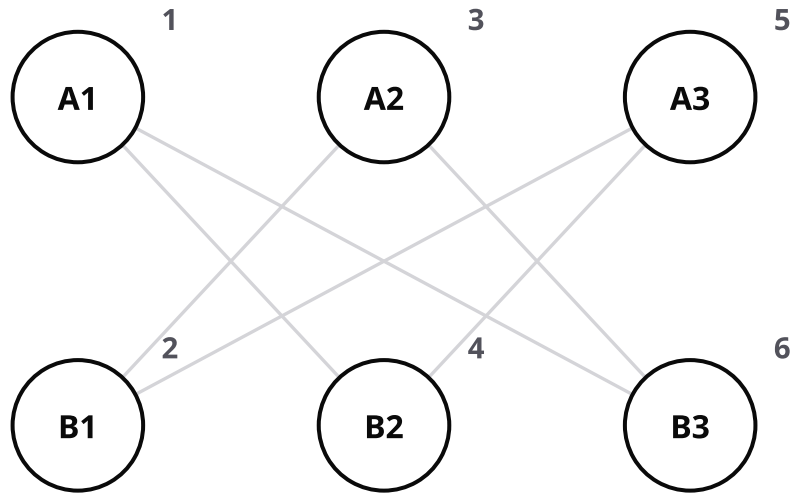
Bảng tổng hợp 5 thuật toán

Thuật toán	Kết luận chắc chắn?	Độ phức tạp (lý thuyết / trong app)	Giới hạn n
Greedy	Không	$O(n \cdot k+m)$ / $O(n \cdot (n+k))$	320
Welsh–Powell	Không	$O(n \log n+n \cdot k+m)$ / $O(n^2+n \cdot k)$	320
DSATUR	Không (đúng trên đồ thị 2 phía)	$O(n^2+m)$ / $O(n^3+n \cdot k)$	320
Backtracking	Có	$O((n+1) \cdot k^n \cdot (\Delta+1))$ / $O(n^2 \cdot k^n)$	9 (k màu) / 10 (χ)
Exhaustive search	Có	$O(k^n \cdot (n+m))$ / $O(k^n \cdot n^2)$	$k^n \le 200.000$ (~6, χ)

Vì sao heuristic có thể thất bại

- Đồ thị vương miện 6 vertex: nhóm A = {A1,A2,A3}, nhóm B = {B1,B2,B3}; Ai kỵ mọi Bj trừ Bi → hai phía, χ = 2.
- Thứ tự nhập cố ý xen kẽ A1,B1,A2,B2,A3,B3 — đây là thứ tự "xấu" cho greedy/Welsh–Powell.
- DSATUR tránh được vì luôn ưu tiên vertex bị ràng buộc nhiều nhất, không phụ thuộc thứ tự nhập.
- Đây là ví dụ chung dùng xuyên suốt report, demo và video — số bước phải khớp nhau ở mọi nơi.

Đồ thị vương miện 6 vertex (χ=2)



Ví dụ chung: thiết lập dữ liệu

DEMO TRỰC TIẾP

- 6 loài: A1, B1, A2, B2, A3, B3 (đúng thứ tự nhập).
- Edge: Ai–Bj với i ≠ j (6 edge).
- Chế độ "Tô bằng k màu cho sẵn", k = 2.
- JSON dán trực tiếp vào tab "Chỉnh JSON" — dữ liệu cập nhật ngay, không cần nút áp dụng.

JSON ví dụ chung

```
{
  "nodes": ["A1","B1","A2",
            "B2","A3","B3"],
  "edges": [
    {"from":"A1","to":"B2"},
    {"from":"A1","to":"B3"},
    {"from":"A2","to":"B1"},
    {"from":"A2","to":"B3"},
    {"from":"A3","to":"B1"},
    {"from":"A3","to":"B2"}],
  "colors": 2
}
```

Quay lui — bước 1 đến 9: đi tới bế tắc

DEMO TRỰC TIẾP

Bước	Vertex	Thao tác	Mảng màu (A1 B1 A2 B2 A3 B3)
1	—	Khởi tạo, thứ tự A1 B1 A2 B2 A3 B3	– – – – – –
2	A1	Gán màu 0	0 – – – – –
3	B1	Gán màu 0 (không kề A1)	0 0 – – – –
4	A2	Loại màu 0 (kề B1) → thử tiếp	0 0 – – – –
5	A2	Gán màu 1	0 0 1 – – –
6	B2	Loại màu 0 (kề A1) → thử tiếp	0 0 1 – – –
7	B2	Gán màu 1	0 0 1 1 – –
8	A3	Loại màu 0 (kề B1)	0 0 1 1 – –
9	A3	Loại màu 1 (kề B2) → hết màu, bế tắc	0 0 1 1 – –

Quay lui — bước 10 đến 20: quay lui và hoàn tất

DEMO TRỰC TIẾP

Bước	Vertex	Thao tác	Mảng màu (A1 B1 A2 B2 A3 B3)
10	—	Backtrack: bỏ B2=1 (hết màu khả thi)	0 0 1 – – –
11	—	Backtrack: bỏ A2=1	0 0 – – – –
12	—	Backtrack: bỏ B1=0 — gốc của bế tắc	0 – – – – –
13	B1	Gán màu 1 (đổi màu)	0 1 – – – –
14–19	…	A2=0, B2=1, A3=0, B3=1 — không còn bế tắc	→ 0 1 0 1 0 1
20	—	Hoàn tất: tô thành công, 20 bước, 6 lần pruning	0 1 0 1 0 1

So sánh 5 thuật toán trên ví dụ chung

DEMO TRỰC TIẾP

- Mỗi thuật toán có checkbox riêng để bật/tắt khỏi so sánh; dropdown "Đồng bộ" đồng bộ các lane: Độc lập / % tiến độ / Giai đoạn / Số bước.
- Mỗi lane có thanh trượt "Bước {tên thuật toán}" riêng; bảng kết quả có cột "Trung vị ms", và nút "Đo hiệu năng theo n" mở biểu đồ benchmark.

Thuật toán	Số bước	Phép toán	Kết quả
Greedy	7	5	"Chưa tìm được" — hết màu ở A3
Welsh–Powell	7	5	Giống Greedy — mọi vertex cùng degree 2
DSATUR	14	6	Tô thành công: [0,1,0,1,0,1]
Backtracking	20	21	Tô thành công: [0,1,0,1,0,1]
Exhaustive search	69	67	Tô thành công: nghiệm đầu tiên ở lá thứ 22

Chế độ "Tìm sắc số χ(G)" trên ví dụ chung

DEMO TRỰC TIẾP

Thuật toán	Số màu dùng	Số bước	Kết quả
Greedy	3	8	Hợp lệ nhưng chưa tối ưu
Welsh–Powell	3	8	Hợp lệ nhưng chưa tối ưu
DSATUR	2	14	Tối ưu
Backtracking	2	22	χ = 2, chính xác
Exhaustive search	2	78	χ = 2, chính xác

Giao diện Player (1/2) — điều hướng và nhập liệu

DEMO TRỰC TIẾP

- Thanh bên: tìm bài theo chương sách, theo kỹ thuật thiết kế, hoặc theo tên bài toán (Ctrl+K).
- Nút chuyển nhanh giữa 5 cách giải của cùng bài toán: Greedy, Welsh–Powell, DSATUR, Backtracking, Exhaustive search.
- Trình nhập dữ liệu: JSON, bộ sinh ngẫu nhiên theo seed, đồ thị mẫu kinh điển (Vương miện, K_n, C_n, K(p,q), Petersen, Grötzsch).
- Khung đồ thị: màu từng vertex ghi bằng số trên nhãn, vertex đang xét được làm nổi bật.

Giao diện Player (2/2) — điều khiển và 2 cách xem

DEMO TRỰC TIẾP

- Khung mã giả: dòng đang chạy được tô đậm.
- Thẻ "Giải thích bước": lời giải thích tiếng Việt cho từng bước.
- Thẻ "Trạng thái": hiển thị Giai đoạn hiện tại của thuật toán (place / reject / backtrack / done).
- Thanh điều khiển: tiến / lùi / tua / tốc độ, có phím tắt.
- Nút chuyển view giữa "Đồ thị" và "Cây trạng thái" — cây đánh dấu riêng các nhánh bị cắt (pruned).
- Toàn giao diện chỉ dùng trắng/đen/xám — trạng thái phân biệt bằng độ đậm và viền, không phụ thuộc màu sắc.

Dữ liệu không hợp lệ bị từ chối ngay khi nhập

DEMO TRỰC TIẾP

- Validation chạy ngay khi dán JSON, không cần bấm nút áp dụng.
- colors = 0 vi phạm ràng buộc $1 \le k \le 10$ → app báo lỗi, không cho chạy.
- Mục đích: tránh crash hoặc kết quả vô nghĩa khi người dùng nhập sai.

JSON không hợp lệ

```
{
  "nodes": ["A"],
  "edges": [],
  "colors": 0
}
```

Trang Testcase — 22 ca, xuất CSV/JSON

DEMO TRỰC TIẾP

- 22 testcase, chia 6 nhóm: thông thường, biên, nhỏ, lớn, đặc biệt, không hợp lệ.
- Chạy toàn bộ bằng một nút, kết quả "Đạt 22/22".
- Hầu hết các nhánh thuật toán được phủ bằng 22 file testcase này (có nghiệm, vô nghiệm, χ đã biết, vượt giới hạn...).
- Riêng nhánh "heuristic bế tắc dù đồ thị có nghiệm" được phủ bằng 1 unit test của greedy — chưa có file testcase riêng.
- Xuất kết quả CSV/JSON để đưa trực tiếp vào report.

Đồ thị lớn và giới hạn an toàn của thuật toán chính xác

DEMO TRỰC TIẾP

- Sinh ngẫu nhiên n = 300, p = 0,1, seed = 1, chế độ χ(G).
- Ba heuristic vẫn chạy ra kết quả (không ghi trace vì n > 10) — DSATUR dùng ít màu nhất.
- Backtracking/exhaustive search báo "Không áp dụng" thay vì treo trình duyệt.

Thông báo thật của app (precondition)

```
Không áp dụng: Quay lui tìm χ(G)
chỉ an toàn với đồ thị ≤ 10 đỉnh
khi giữ thứ tự duyệt đã cho (đồ
thị hiện có 300 đỉnh).
```

Tiến trình phát triển

#	Giai đoạn	Kết quả
1–3	Phân tích đề → khảo sát sách → thiết kế kiến trúc	Note cấu trúc, danh mục ~60 thuật toán, kiến trúc đã chốt
4–5	Bản đầu (cái túi 0/1) → deploy	PR #1 (11/10); chuyển sang Cloudflare Pages
6–8	Nền tảng catalog → 5 nhóm song song → tích hợp	PR #2–#8, merge 12/10
9–10	Domain riêng → chiến lược report	algorithm-visualizer.hieutk.dev (17/10)
11	Nâng cấp tô màu: $\chi(G)$, đồ thị lớn, 22 testcase	3 vòng CHANGES REQUESTED → PR #12, merge 18/10

Dùng AI trong xây dựng — công cụ và nguyên tắc

- **Toàn bộ mã nguồn do AI sinh: Claude Code (Anthropic) và Codex (OpenAI).**
- Sinh viên KHÔNG tự viết code và KHÔNG tự review code.
- Nguyên tắc review chéo khác hãng: việc do Claude Code làm thì Codex review, và ngược lại.
- Reviewer chỉ nhận bản diff cố định theo commit, không được sửa code; lỗi chặn gửi lại kèm yêu cầu test hồi quy fail trên code cũ.

Việc sinh viên trực tiếp làm

Việc
Đặt yêu cầu, phạm vi (toàn bộ sách, không chỉ 1 bài)
Chọn công nghệ, phong cách UI (shadcn, trắng đen)
Chọn deploy Cloudflare Pages, domain riêng
Yêu cầu các mô hình AI tranh luận chiến lược
Merge PR sau khi review chéo đạt (mergedBy: tkhieu)
Xử lý agent bị chặn (duyệt ngân sách, đăng nhập lại)

Prompt mẫu (trích nguyên văn)

Prompt thiết kế giao diện (nguyên văn, giữ nguyên chính tả gốc)

“Hãy chọn 1 đề tài mà bạn nghĩ là hợp lý,. build theo kiến trúc khuyến nghhị, build cho tới khi xong có thể demo đựoc. UI chọn shadcn ui tông màu sáng. trắng đen.”

Prompt giao việc nhóm B5 (trích đoạn đầu, log c6dcc10b)

“/goal IMPLEMENT group B5 (tsp, assignment, n-queens, hamiltonian-circuit, subset-sum, graph-coloring (bai So thu), matrix-chain, lcs…”

Prompt review lượt 2 (nguyên văn, log 04cbfb0d)

“REVIEW (cross-vendor; fixes by Codex, original by Claude Code) of catalog group B5… Previous review FAILED with 8 blocking issues…”

Các sự cố tiêu biểu khi dùng AI và cách xử lý

#	Lỗi AI gây ra	Ai phát hiện	Cách sửa
1	Checker tự tham chiếu + heuristic khẳng định sai "vô nghiệm"	Review chéo (Codex)	Oracle existsColoring độc lập + cờ heuristic
2	Nhánh cận đánh dấu sai nút tốt nhất là "bị cắt"	Review chéo (Claude Code)	8 test hồi quy, fail trên code cũ
3	Lời giải tham chiếu tự tham chiếu (B1, B4, B5)	Review chéo	Thay oracle độc lập (đếm, mô phỏng, Held-Karp…)
4	136/184 file viết dồn 1 dòng (>1.000 ký tự/dòng)	Review chéo	Format lại — chỉ sửa dòng cần thiết, tránh đụng file chung

Bài học rút ra khi dùng AI

- **AI làm tốt:**
- dựng nhanh hệ thống lớn (47 bài toán, 126 cách giải trong ~22 giờ), giữ đúng kiến trúc chung, tự viết test.
- **AI hay sai:**
- test tự tham chiếu; trace minh họa sai dù đáp án đúng; "làm cho có"; code khó đọc.
- **Vẫn phải do người kiểm:**
- phạm vi và tính trung thực của report; chạy tay để hiểu thuật toán; quyết định merge.

Thiết kế thực nghiệm — bộ testcase

- Viết tay (ca nhỏ, đáp án dễ thấy) + dựng có chủ đích (đồ thị kinh điển đã biết χ) + sinh ngẫu nhiên theo seed.
- App tự hạ xác suất edge p để kỳ vọng số edge ≤ 7.200, từ chối nếu số edge thực vượt 8.000.
- 22 file testcase, xuất qua lệnh export của app ra testcases/graph-coloring/.

6 nhóm testcase

Nhóm	Số file	Mục đích
Thông thường	5	Ca điển hình
Biên	3	Min/rỗng
Nhỏ	2	Kiểm tay được
Lớn	5	n đến 310, kiểm hiệu năng
Đặc biệt	5	K_n, Petersen, Grötzsch…
Không hợp lệ	2	k=0, mode sai

Hiệu năng trên đồ thị ngẫu nhiên lớn (G(n, p=0,1), 5 seed, trung vị)

n	Greedy (ms / màu)	Welsh–Powell (ms / màu)	DSATUR (ms / màu)
50	0,098 / 5	0,241 / 4	1,173 / 4
100	0,279 / 7	0,692 / 6	7,958 / 6
200	1,359 / 12	3,181 / 10	64,816 / 8
300	3,579 / 14	7,363 / 13	203,489 / 11

Đánh giá: tính đúng, trực quan, hạn chế

- **Tính đúng:**
- "Đạt 22/22"; 314/314 e2e pass trên chính domain https://algorithm-visualizer.hieutk.dev; 825 unit test.
- Backtracking: so với oracle độc lập existsColoring, trên nhiều seed/kích thước. Exhaustive search: so với backtracking trên nhiều input, cộng kiểm trực tiếp bằng oracle χ(G) trên vài đồ thị nhỏ (vương miện, chu trình).
- **Trực quan:**
- mỗi bước có mã giả + giải thích + trạng thái đồ thị; lùi được bất kỳ bước; state-space tree thấy rõ lúc backtrack.
- **Hạn chế:**
- thuật toán chính xác chỉ chạy đồ thị nhỏ; với n > 12 không biết χ thật; heuristic không bảo đảm; toàn bộ code do AI sinh, có thể còn lỗi chưa phát hiện.

Hướng phát triển & mở rộng: toàn bộ sách Levitin

DEMO TRỰC TIẾP

- **Hướng phát triển:**
- thêm nhánh cận (clique + DSATUR) để tìm χ(G) trên đồ thị lớn hơn; thêm biến thể (tô màu edge, list coloring); nhập đồ thị bằng vẽ tay.
- **Mở rộng đã có (≈30s):**
- cùng nền tảng Player/So sánh/Testcase phục vụ 47 bài toán, 126 cách giải — phần lớn theo sách Levitin (Dijkstra, LCS, TSP, n-hậu…), một số ngoại lệ ngoài sách được ghi rõ trong app (DSATUR, Hungarian). Phần chấm điểm chính vẫn là bài Tô màu đồ thị.

Cảm ơn cô đã xem

- Hạn chế: thuật toán chính xác chỉ chạy đồ thị nhỏ; heuristic không bảo đảm tìm nghiệm.
- Mã nguồn được sinh bằng AI (Claude Code, Codex), kiểm chứng bằng review chéo và test độc lập — chi tiết trong report.
- Demo trực tuyến: algorithm-visualizer.hieutk.dev